

Siano A e B due **insiemi**. Il **coefficiente di Jaccard** è definito come:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Il coefficiente di Jaccard restituisce sempre un numero compreso tra 0 ed 1, perciò possiamo usarlo come funzione di ordinamento per il risultato della nostra query.

Infatti abbiamo che:

- $J(A, B) = 0 \implies$ sono totalmente differenti.
- $J(A, B) = 1 \implies$ sono uguali.

Se poniamo $A = \text{query}$ e $B = \text{document}$, possiamo misurare la "rilevanza" del documento B rispetto alla query A .

La misura di **Jaccard** non è una metrica, in quanto non vale che:

- $J(A, A) = 0$
- $J(A, B) = J(A, X) + J(X, B)$ (disuguaglianza triangolare)

Abbiamo come proprietà solo la **simmetria**.

Non possiamo quindi definire uno **spazio metrico**.

Problematiche

Questo coefficiente purtroppo non tiene conto della **frequenza** in cui i termini appaiono in un documento: tiene solo conto di quali termini della query appaiono nel documento.

Generalmente la frequenza di un termine in un documento è una sorte di **indice di rilevanza** del documento.

Esempio: Abbiamo due documenti D_1 e D_2 . La query è $B = \text{term}X$. Supponiamo che il termine $\text{term}X$ sia presente in entrambi i documenti, con frequenza relativamente 3 volte e 88 volte. Se la dimensione è la stessa, allora avremo che $J(B, D_1) = J(B, D_2)$ anche se in teoria il documento D_2 è più significativo.

Un altro problema è la **dimensione** del documento. Infatti più un documento è grande, più esso verrà **penalizzato** nello scoring.

Un accorgimento è quindi quello di normalizzare per la **radice** dell'unione degli insiemi, così da penalizzare un pò meno i documenti grandi.

$$J'(A, B) = \frac{|A \cap B|}{\sqrt{|A \cup B|}}$$

Bag of words model - Term Frequency (TF)

Un modo alternativo al **Jaccard coefficient** che tiene anche conto della **frequenza dei termini** per determinare la rilevanza dei documenti, è il semplice modello a **bag of words**.

Per rappresentare la collezione usiamo una semplice **Binary Term-Document Incidence Matrix** dove però invece di avere 0-1, abbiamo la **frequenza** del termine nel documento.

Perciò, sia la **term frequency** $tf_{t,d}$ il numero di volte in cui il termine t appare nel documento d : la sua **frequenza**.

Definiamo una matrice $terms \times docs$, dove nella cella (i, j) è presente il valore $tf_{i,j}$. Questa matrice è anche detta **Count Matrix** o **Term-Frequency Matrix**.

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	157	73	0	0	0	1	
Brutus	4	157	0	2	0	0	
Caesar	232	227	0	2	1	0	
Calpurnia	0	10	0	0	0	0	
Cleopatra	57	0	0	0	0	0	
mercy	2	0	3	8	5	8	
worser	2	0	1	1	1	5	
...							

I documenti sono quindi rappresentati come un insieme di **vettori** in $N^{|V|}$, dove V è l'insieme dei termini (o vocaboli).

Possiamo sfruttare queste **term frequency** basandoci sull'idea che maggiore è il valore $tf_{t,d}$ maggiore il documento d è **rilevante** rispetto al termine t .

Questo metodo è **Naive**, in quanto non tiene conto della **semantica** della query.

Infatti, se cerco:

- "Il gatto morde il cane"
- "Il cane morde il gatto"

tale modello restituisce lo stesso risultato, attribuendo lo stesso voto agli stessi documenti.

Osserviamo però che in realtà la rilevanza di un documento d rispetto al termine t non cresce **linearmente** rispetto alla sua frequenza $tf_{t,d}$. Se un termine è presente 10 volte in un documento A e una volta in un documento B , non è vero che A è esattamente 10 volte più rilevante rispetto a B .

Mi aspetto che affinché A sia 10 volte più rilevante rispetto a B deve essere che $tf_{t,A}$ è **ordini di grandezza** più grande rispetto a $tf_{t,B}$.

Ancora, se $tf_{t,A} = 300,000$ e $tf_{t,B} = 300,012$ mi aspetto che A e B abbiano praticamente la stessa rilevanza.

Perciò, applichiamo una **scalatura non lineare** nelle sequenze, così da misurare le frequenze solamente tra **scale di risoluzione significative**.

$$w_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Definiamo infine lo **score** di un documento d rispetto a una query q come

$$\text{tf-score}(q,d) = \sum_{t \in q \cap d} w_{t,d}.$$

Se tale coefficiente è pari a 0 vuol dire che nessun termine di q è presente in d .

TF-IDF Weight

Consideriamo la query "the dog". Come risultato della query otterrò tantissimi documenti inutili alla mia ricerca, per il semplice motivo che è presente il termine "the" un numero elevato di volte.

Un primo approccio semplice che abbiamo già incontrato è la **Inverted Index**. Questo metodo però non è molto elegante, perché alcune stopwords potrebbero risultare utili ai fini della mia query. Per esempio, potrei cercare nella mia query "come si dice in inglese l'articolo 'un'?". Eliminando le stopwords, gli unici termini utili risulterebbero "come, dice, inglese, articolo".

Un'idea più elegante è invece che i termini che sono più **rari** in una lingua sono più **informativi** in un documento. Più semplicemente si può pensare che un termine che occorre più volte sarà **meno informativo** di un termine **più raro**.

Se per esempio la mia query è "Phenethylamine", sicuramente il termine "Phenethylamine" è decisamente più utile ai fini della ricerca, anziché "the". E questo accade perché "Phenethylamine" è decisamente più raro nella lingua inglese rispetto a "the".

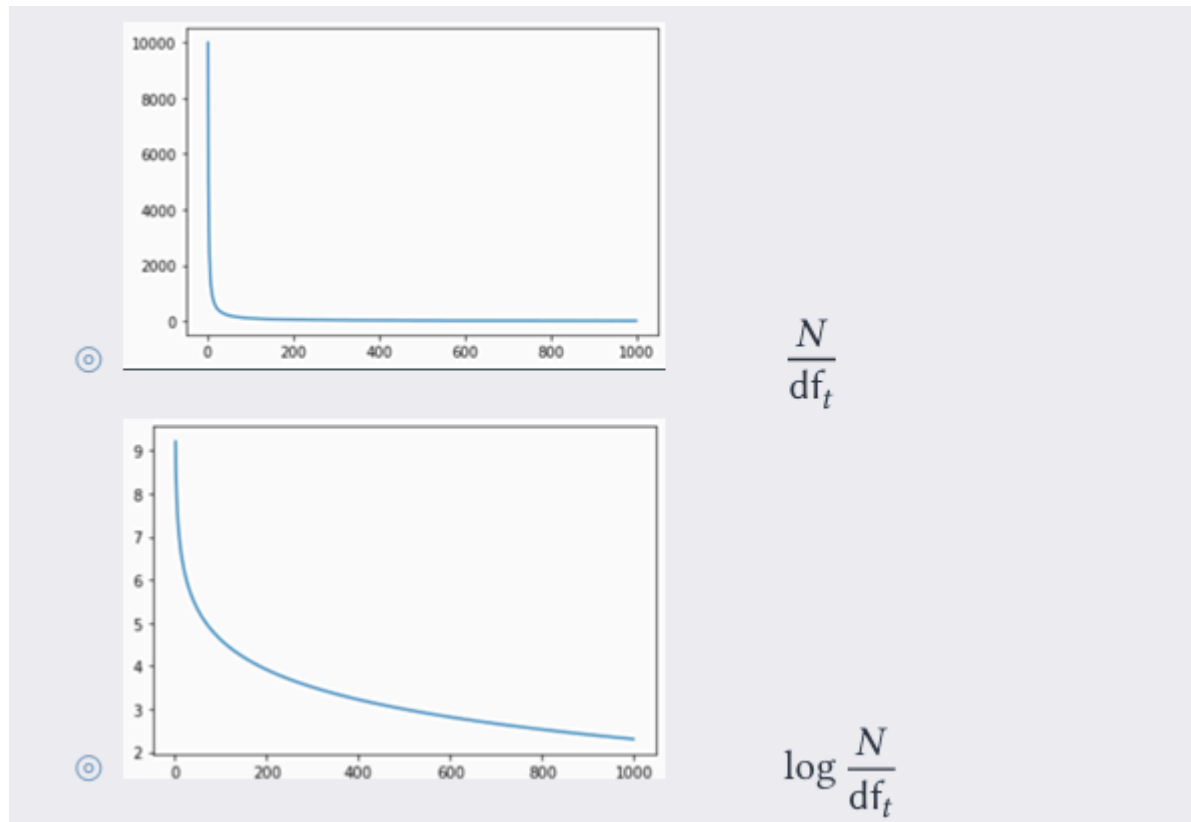
E' ragionevole quindi **pensare** i termini in base alla loro **rarietà**, sfruttiamo di nuovo la loro **document frequency**.

Definiamo quindi la **informativeness** di un termine come

$$\text{idf}_t = \log_{10} \frac{N}{\text{df}_t}$$

dove df_t è la **document frequency** del termine t nella mia collezione, ed N è il numero di documenti.

Si preferisce applicare una trasformazione non lineare a N/df_t per avere una curva di decrescita **meno ripida**.



A questo punto definiamo il **peso** di un termine t rispetto a un documento d combinando la sua **Bag of Words model - Term Frequency tf** e la sua **informativeness**.

$$w_{t,d} = (1 + \log(\text{tf}_{t,d})) \cdot \log\left(\frac{N}{\text{df}_t}\right) = \text{tf-weight} \cdot \text{idf-weight}$$

Tale valore $w_{t,d}$ è anche noto come **tf-idf weight**.

- **tf** = term frequency,
- **idf** = informative document frequency.

Vector Space Model

Abbiamo visto in precedenza che è utile rappresentare i documenti come:

- Un **Binary Term-Document Incidence Matrix** in $\{0, 1\}^{|V|}$
- Un **Bag of words model - Term Frequency tf** in $N^{|V|}$

Alla luce di quanto visto finora, è più ragionevole definire una matrice di valori **reali**, dove alla cella (i, j) è presente il **peso ft-idf** $w_{i,j}$.

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	5.25	3.18	0.0	0.0	0.0	0.35	
Brutus	1.21	6.10	0.0	1.0	0.0	0.0	
Caesar	8.59	2.54	0.0	1.51	0.25	0.0	
Calpurnia	0.0	1.54	0.0	0.0	0.0	0.0	
Cleopatra	2.85	0.0	0.0	0.0	0.0	0.0	
mercy	1.51	0.0	1.90	0.12	5.25	0.88	
worser	1.37	0.0	0.11	4.15	0.25	1.95	
...							

Tale matrice è anche detta **score matrix**, e ogni documento è ora rappresentato con un vettore in $R^{|V|}$.

Grazi a questa rappresentazione, abbiamo che i nostri documenti ora sono dei **punti** o **vettori** in uno spazio $|V|$ -dimensionale, dove i termini rappresentano le **dimensioni** o **assi** di questo spazio.

La lunghezza di ogni coordinata di un vettore che rappresenta un documento è quindi il peso tf-idf rispetto alla coordinata.

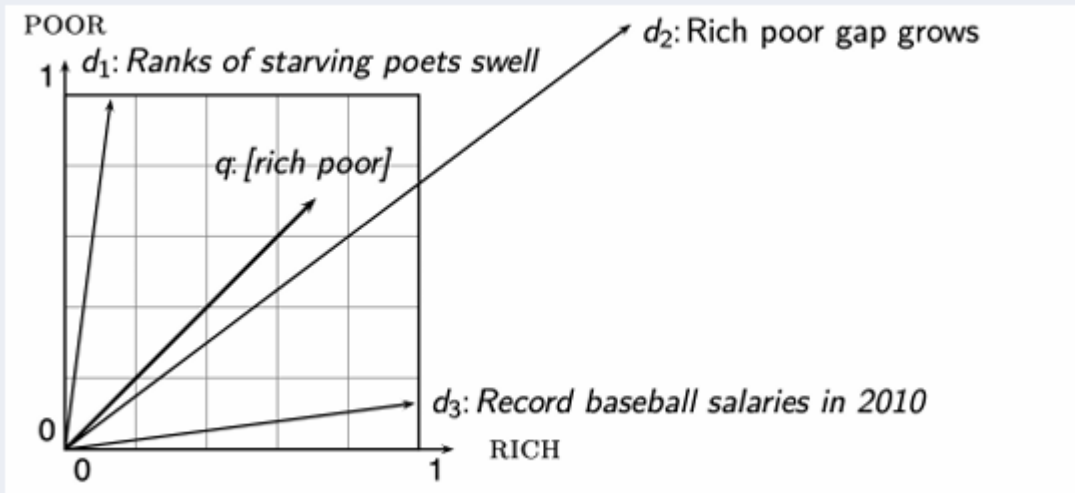
Osservare che tale spazio può avere un numero di dimensioni veramente esorbitante: anche decine di milioni se pensiamo al web.

In ogni caso, quando le dimensioni sono tante, i vettori sono per lo più **sparsi** (con molte entries nulle).

Secondo questo modello, possiamo rappresentare anche le **query come vettori**. A questo punto potremmo definire un **rank** dei documenti in base alla loro *prossimità* al vettore-query.

Ma quale matrice di **distanza** conviene usare? Per esempio la classica **distanza euclidea** potrebbe funzionare bene se i vettori avessero **magnitudo** simili.

Purtroppo però i documenti hanno migliaia di termini in più rispetto alle query (sono **molti meno sparsi**) perciò potrebbero essere davvero parecchio distanti dal vettore-query, anche se rilevanti alla fine del ranking.



Per rendere meglio l'idea, consideriamo un vettore-documento d . Da esso creiamo il vettore d' , composto dalla concatenazione di due o più volte d stesso. Semanticamente i due vettori d e d' sono la stessa cosa, però se usiamo la distanza euclidea come misura di similarità capiterà che uno è decisamente migliore rispetto all'altro.

Un modo migliore per misurare la similarità tra i vettori di questo spazio è quindi la loro **direzione**. L'idea è che se due vettori puntano nella stessa direzione nello spazio allora semanticamente voglio dire la stessa cosa.

Perciò, più la direzione di un vettore- documento è simile a quella del vettore-query, più esso sarà rilevante ai fini del ranking.

Per calcolare la differenza di direzione possiamo sfruttare l'**angolo** tra vettori.

Osservare che, dato un angolo $\alpha \in [0, \pi]$, la funzione $\cos(\alpha)$ sarà **decrescente**. Perciò ordinare i documenti in senso **decrescente rispetto all'angolo** è equivalente ad ordinarli in senso **crescente rispetto al coseno**.

E' inoltre possibile calcolare il coseno tra due vettori in maniera efficiente semplicemente **normalizzandoli**. Dati due vettori unitari, il loro coseno equivale al loro **prodotto scalare**.

Normalizzare un vettore

Prendiamo un generico vettore $v \in \mathbb{R}^n$.

Il vettore **normalizzato** di v equivale a v stesso ma scalato in modo tale da avere **lunghezza** pari ad 1.

Per ottenerlo, basterà dividere ogni componente di v con la sua **norma euclidea**, o **norma 2**.

$$\|v\|_2 = \sqrt{\sum_{i=1}^n v_i^2}$$

Perciò avremo che

$$\left\| \frac{v}{\|v\|_2} \right\|_2 = 1$$

Prodotto scalare

Il **prodotto scalare** (o **dot product**) tra due vettori $u, v \in \mathbb{R}^n$ è definito come

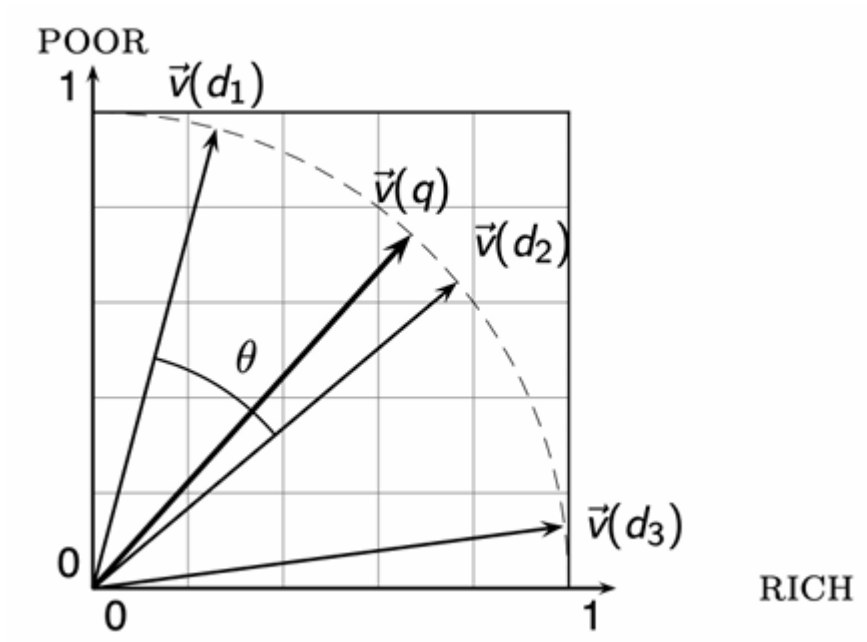
$$u \cdot v = \sum_{i=1}^n u_i \cdot v_i$$

Come risultato di questa normalizzazione avremo che:

1. Tutti i punti saranno poggiati su di una (**iper**-) **sfera unitaria**.
2. I punti d e d' (controesempio) sono ora **identici**.
3. Possiamo calcolare il coseno di due vettori unitari q (query) e d (document) come il loro prodotto scalare:

$$SIM(q, d) = \cos(\text{angle}(q, d)) = \frac{q}{\|q\|} \cdot \frac{d}{\|d\|} = \frac{\sum_{i=1}^{|V|} q_i \cdot d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \cdot \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

Tale misura di similitudine è anche detta **cosine similarity**.



```
import LinearAlgebra: normalize
global COLLECTION
global K = 10 # number of result's document

function tf_score(term::String, doc::Union{Document, String,
Vecotr{String}})::Float64
```

```

    if count(term, doc) > 0
        1.0 + log10(count(term, doc))
    else
        0.0
    end
end

# ovviamente è più efficiente **precomputare** gli informative document
# frequency dei vari termini
function idf_score(term::String, collection::Collection)::Float64
    N = length(collection)
    doc_frequency = findall(doc -> term ∈ doc, collection) |> length
    log10(N/doc_frequency)
end

function tf_idf_weights(term::String, document::Union{Document, String,
Vecotr{String}})
    global COLLECTION
    tf_score(term, document) * idf_score(term, COLLECTION)
end

function as_vecotr(d::Union{Document, String,
Vecotr{String}})::Vector{Float64}
    global COLLECTION
    result = Float64[]
    for term ∈ terms(COLLECTION)
        add!(result, tf_idf_weights(term, d))
    end
    result
end

function cosine_sim(p1::T, p2::T)::Float64 where T <: Vector{Float64}
    p1 = normalize(p1)
    p2 = normalize(p2)
    sum(p1 .* p2)
end

function CosineScore(query::Union{String, Vecotr{String}})
    global COLLECTION, K
    query_vec = as_vecotr(query)
    score = Tuple{Document, Float64}[]
    for document ∈ COLLECTION
        doc_vec = as_vecotr(document)
        s = cosine_sim(doc_vec, query_vec)
        push!(score, (document, s))
    end
end

```



```

    sort!(score, by = x -> x[2], rev=true) # sort by score
    return score[1:K]
end

```

```

COSINESCORE( $q$ )
1  float Scores[ $N$ ] = 0
2  float Length[ $N$ ]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6          do Scores[ $d$ ] +=  $w_{t,d} \times w_{t,q}$ 
7  Read the array Length
8  for each  $d$ 
9  do Scores[ $d$ ] = Scores[ $d$ ] / Length[ $d$ ]
10 return Top  $K$  components of Scores[]

```

Variants of Weighting

Nel modello **Bag of Words -Term Frequency tf** usiamo come **term frequency** solamente il numero di documenti in cui un termine t appare in un dato documento d .

Analizziamo ora alcune varianti interessanti. Prima però definiamo:

- $|d|$ il numero di termini **distinti** nel documento d .
- $|c|$ il numero di documenti in una collezione c .
- $tf_{t,d}$ il numero di volte in cui un termine t appare nel documento d .
- $N(d) = \sum_t tf_{t,d}$ il numero di occorrenze dei termini nel documento d .
- $N_{avg}(d) = \frac{N(d)}{|d|}$ la **media** del numero di occorrenze dei termini nel documento d .
- $adl(c) = \frac{\sum_{d \in c} N(d)}{|c|}$ il numero **medio** di occorrenze dei termini nei documenti della collezione d .
- $ndl(d, c) = \frac{N(d)}{adl(c)}$ il numero di occorrenze dei termini in d **normalizzato** (tra 0 e 1).

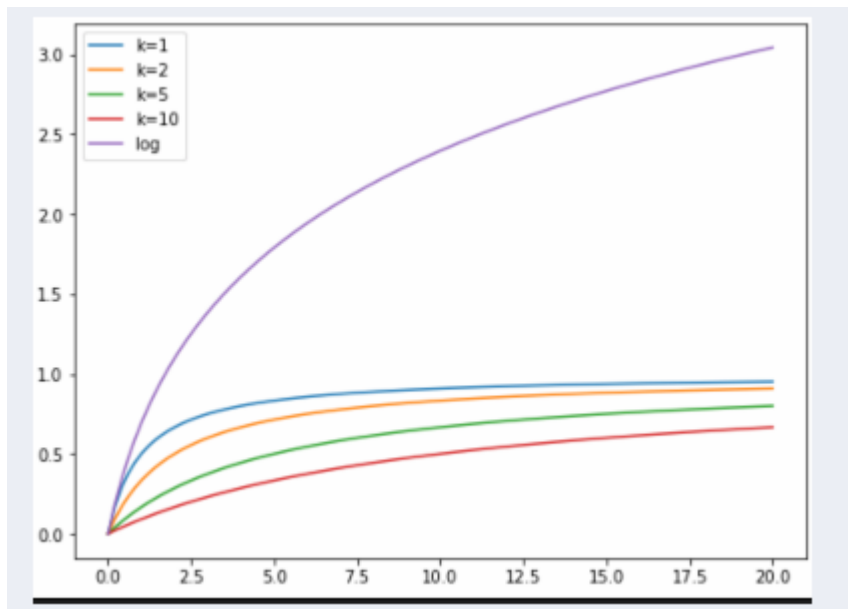
Osservazioni importanti

1. $N(d)$ è la **lunghezza** (inteso come numero di parole) del documento d . Questo è differente da $|d|$ perché conta anche i duplicati ($|d|$ conta solo quelli distinti).
2. Alla luce del punto (1) abbiamo che $adl(c)$ è la **lunghezza media** dei documenti nella collezione c .

natural	$TF_{total}(t, d)$	$n(t, d)$
boolean	$TF_{bool}(t, d)$	$\begin{cases} 1 & \text{if } n(t, d) > 0 \\ 0 & \text{otherwise} \end{cases}$
sum	$TF_{sum}(t, d)$	$\frac{n(t, d)}{N(d)}$
max	$TF_{max}(t, d)$	$\frac{n(t, d)}{\max_{t'} n(t', d)}$
augmented	$TF_{aug}(t, d)$	$0.5 + \frac{0.5 \cdot n(t, d)}{\max_{t'} n(t', d)}$
log	$TF_{log}(t, d)$	$\log(1 + n(t, d))$
log avg	$TF_{logavg}(t, d)$	$\frac{\log(1 + n(t, d))}{\log(1 + na(d))}$
frac	$TF_{frac}(t, d; k)$	$\frac{n(t, d)}{n(t, d) + k}$
BM25	$TF_{BM25}(t, d, c; k, b)$	$\frac{n(t, d)}{n(t, d) + k(b \cdot ndl(d, c) + (1 - b))}$

Osservazioni sulle funzioni

1. Le funzioni TF_{total} , TF_{sum} , TF_{max} si basano tutte sull'assunzione di **indipendenza** delle occorrenze. La **Term Frequency** continua a crescere della stessa quantità ma man mano che trovo le occorrenze nel documento d , **indipendentemente** da quante ne ho trovate in precedenza.
2. La funzione TF_{total} non è **normalizzata** rispetto alla grandezza del documento, perciò ha una forte tendenza a preferire i documenti più grandi. Infatti, supponiamo di avere una collezione di documenti tutti con la stessa **percentuale** di occorrenze di un termine t . Con TF_{total} quelli più grandi sono preferiti a quelli più piccoli, anche se in proporzione sono tutti uguali.
3. La funzione TF_{sum} invece **normalizza** rispetto alla **lunghezza** del documento.
4. Invece TF_{max} è una via di mezzo tra TF_{total} e TF_{sum} : per la stessa percentuale di **term frequency** preferisce i documenti con grandezza maggiore, ma non così tanto come TF_{total} .
5. La funzione TF_{frac} dà un rapporto più basso a frequenze basse, per poi andare a regolarizzarsi per frequenze molto grandi.
6. Infine la funzione TF_{log} applica una funzione logaritmo per mettere in paragone solamente le **scale di grandezza** delle frequenze.



Variants of idf weighting

Definiamo:

- $n(t, c) = \sum_{d \in c} TF_{bool}(t, d)$ ovvero il numero di documenti nella collezione c in cui il termine t appare.
- $|c|$ il numero di documenti in una **collezione** c .

total	$idf_{total}(t, c)$	$-\log n(t, c)$
sum	$idf_{sum}(t, c)$	$-\log \frac{n(t, c)}{ c }$
smooth sum	$idf_{smooth}(t, c)$	$-\log \frac{n(t, c) + 0.5}{ c + 1}$
prob	$idf_{prob}(t, c)$	$\max \left(0, -\log \frac{n(t, c)}{ c - n(t, c)} \right)$
smooth prob	$idf_{smoothprob}(t, c)$	$\max \left(0, -\log \frac{n(t, c) + 0.5}{ c - n(t, c) + 0.5} \right)$